Bioinformatics, 2026, , 1–6

Applications Note

PyFgsea: A Rust-Powered, fgseaMultilevel-Aligned GSEA Framework with Rolling-Window Enrichment Along Single-Cell Trajectories

Kuanghao Wang[1] and Hong Shi[1,*]

[1]Institute of Primate Translational Medicine, Kunming University of Science and Technology, Kunming 650500, China
*To whom correspondence should be addressed. Email: shih@kust.edu.cn

Abstract

Summary: **GSEA is a standard approach for pathway interpretation, yet Python ecosystems lack a high-performance implementation aligned with the fgseaMultilevel rare-event estimator target, especially for trajectory-aware rolling-window analysis. PyFgsea achieves numerical alignment with the R fgseaMultilevel reference for normalized enrichment scores (NES; Pearson $r > 0.999$) and significantly reduces runtime compared to existing Python tools. Its stateful rolling-window engine further reduces repeated trajectory-analysis overhead, yielding approximately 1.9-fold end-to-end wall-time speedup in a conservative stress test and up to 7.47-fold component-level acceleration in a narrower 100-window benchmark.**

Availability and implementation: **Source code is available at https://github.com/shayuanxukuang/pyfgsea and via PyPI (`pip install pyfgsea`). An archival snapshot of the code and benchmark data is available on Zenodo (DOI: 10.5281/zenodo.19446446).**

Contact: **shih@kust.edu.cn**

Supplementary information: **Supplementary data are available at Bioinformatics online.**

Keywords: Gene set enrichment analysis; multilevel sampling; rare-event estimation; single-cell trajectories; Rust; PyO3

1. Introduction

Gene set enrichment analysis (GSEA) is widely used to interpret genome-wide expression profiles by quantifying whether a gene set is enriched toward the top or bottom of a ranked list [1]. In Python, commonly used implementations such as GSEApy [2] and BlitzGSEA [3] offer accessible workflows but may differ in runtime, memory, or the behaviour of extreme-tail P-values under multilevel sampling.

The `fgsea` R package provides an efficient multilevel adaptive sampling procedure ("fgseaMultilevel") that targets accurate rare-event P-values without requiring extremely large numbers of permutations [4]. However, a statistically aligned, high-performance implementation has been missing from the Python ecosystem. This gap becomes more pronounced in single-cell trajectory analysis, where enrichment is often desired along pseudotime using rolling windows, resulting in hundreds to thousands of GSEA evaluations.

We present PyFgsea, a high-performance framework built on a Rust backend with PyO3 bindings. To our knowledge, it fills a critical gap by providing: (i) a Python implementation aligned with the fgseaMultilevel rare-event estimator target; (ii) a multi-threaded architecture that substantially reduces runtime and memory footprint; and (iii) a specialized stateful rolling-window runner that minimizes redundant computations for efficient trajectory analysis.

© The Author(s) 2026.

2. Materials and Methods

2.1. Core multilevel workflow

PyFgsea re-implements the fgseaMultilevel rare-event estimator target with the same ES definition, null-pathway construction, and multilevel tail factorization [4]; in the current Rust release, the conditional levels are realized through empirical median thresholding and elite-clone rejuvenation. A complete methodological workflow diagram is provided in Supplementary Figure S7. For a pathway S of size K in a ranked list of N genes with ranked statistics $\{r_j\}$, PyFgsea uses the same weighted preranked enrichment score as in classical GSEA and fgseaMultilevel [1, 4]:

$$ES = \max_{1\le i\le N} \left(P_{\text{hit}}(i) - P_{\text{miss}}(i)\right), \qquad P_{\text{hit}}(i) = \sum_{\substack{j\le i\\ j\in S}} \frac{|r_j|^p}{\sum_{g\in S}|r_g|^p}, \qquad P_{\text{miss}}(i) = \sum_{\substack{j\le i\\ j\notin S}} \frac{1}{N-K}.$$

The rare-event tail probability is then estimated through the same multilevel factorization [4],

$$P(ES \ge ES_{\text{obs}}) \approx P_0 \prod_{l=1}^{L} P(ES \ge c_l \mid ES \ge c_{l-1}),$$

using an initial baseline sample followed by progressively more extreme conditional levels. The current release therefore combines a baseline sample, empirical median-conditioned levels, elite-clone rejuvenation, and budget/precision/floor-based stopping. The Python API adheres to standard GSEA conventions: users provide ranked genes, a GMT gene set collection, and parameters controlling multilevel estimation (e.g., `eps`, `sample_size`, `seed`). PyFgsea differs primarily at the implementation layer (Rust/PyO3 backend, Rayon scheduling, deterministic pathway-local PRNG streams) and in the functional extension of a stateful rolling-window runner for trajectory analysis; Supplementary Table S5 and Section S8 provide a concise same-versus-different summary and fuller algorithmic details.

2.2. Rust backend and parallelism

The computational core is implemented in Rust and exposed to Python via PyO3. Pathway-level evaluations are parallelized across threads using a dynamic work-stealing strategy. To guarantee deterministic reproducibility across thread counts under matched inputs and a fixed master seed without synchronization blocking, PyFgsea derives independent, deterministic pseudo-random number generator (PRNG) streams for each pathway by hashing a user-provided global master seed with the unique pathway identifier (see Supplementary Section S9). Memory allocations are minimized by reusing buffers across pathways and windows. This design targets both speed and reduced peak RAM for large gene-set collections.

2.3. Rolling-window GSEA for single-cell trajectories

For trajectory analysis, PyFgsea supports rolling-window enrichment along pseudotime. Cells are ordered by pseudotime (e.g., diffusion pseudotime, DPT [5]) computed using Scanpy [6]. A window of fixed size slides along the ordered cells with a user-defined step size. Trajectory termini are handled without padding, and Supplementary Figure S14 additionally audits an explicit terminal-truncation extension. The choice of window and step size governs temporal resolution and statistical stability. As a practical starting range that should be tuned to trajectory length and noise level, we suggest an initial window size spanning 5–10% of the total cells and a step size of 1–2% of total cells. Smaller windows may be overly sensitive to single-cell noise, while excessively large windows risk oversmoothing biologically transient dynamics (a comprehensive parameter sensitivity analysis is provided in Supplementary Figure S8 and Sections S10 and S13). The erythroid illustration in Figure 2 intentionally uses a somewhat larger 500-cell baseline window (approximately 14% of that analyzed trajectory) because this lineage segment is relatively short and noisy, and the figure prioritizes a smoother descriptive

profile over maximal temporal resolution. For each sliding window along the pseudotime-ordered cells, we rank genes by a logFC-like score computed on log1p-normalized expression: $S_g = \mu_{g,in} - \mu_{g,out}$, where $\mu_{g,in}$ and $\mu_{g,out}$ denote mean expression of gene g inside the current window and in all remaining cells, respectively. The ranked list is then used as input to preranked GSEA over a chosen gene-set collection (e.g., MSigDB Hallmark [7, 8]). Within each window, Benjamini–Hochberg adjustment is applied across pathways, so window-level significance corresponds to within-window FDR control. The rolling-window workflow is implemented in `pyfgsea.trajectory.run_trajectory_gsea`, and the repository includes a demonstrative script at `examples/trajectory_demo.py` (release tag: `v0.1.4`).

2.4. Datasets and benchmarking

We evaluated performance and faithfulness on both bulk-like and single-cell settings. For single-cell demonstrations we used Human Cell Atlas (HCA) datasets [9] accessed through `HCAData` [10]. Benchmarking compared PyFgsea with GSEApy [2], BlitzGSEA [3], and the reference R `fgsea` implementation [4] under matched inputs and parameters (see Supplementary Material for detailed parameter alignment).

Benchmarks were conducted on a server equipped with dual Intel Xeon Platinum 8352Y CPUs (64 cores/128 threads, @2.20GHz) and 1.5 TiB RAM, running Ubuntu 22.04.1 LTS. Software versions were Python 3.10.19, R 4.4.3 (fgsea 1.32.2), GSEApy 1.1.11, and BlitzGSEA 1.3.54. To ensure fairness, multithreading was controlled: PyFgsea and Python baselines used 4 threads (via `RAYON_NUM_THREADS=4` or `OMP_NUM_THREADS=4`), while R/fgsea was configured with `nproc=4` (using `BiocParallel`). Each condition was measured in 3 independent repeats using independent processes to minimize cache interference. Runtime and peak memory (PeakRSS) were recorded via `/usr/bin/time -v` or internal timers, reported as mean of 3 repeats in Supplementary Table S4. Detailed performance breakdowns, including thread scaling and rolling-window component analysis, are provided in the Supplementary Material.

3. Results

3.1. Performance and memory footprint

Across representative dataset sizes (spanning small, medium, and large scales), PyFgsea consistently reduced runtime while using substantially less memory than alternative Python implementations (Supplementary Table S4). In the large benchmark (20k genes, 5k gene sets), PyFgsea completed in 1.35 s with 68.0 MB PeakRSS, compared with 18.00 s and 611.4 MB for BlitzGSEA and 189.42 s and 5891.1 MB for GSEApy. These gains are primarily attributable to the Rust backend, pathway-level parallelism, and careful reuse of intermediate buffers.

3.2. Statistical faithfulness to fgseaMultilevel

To assess equivalence to the reference implementation, we compared enrichment scores (ES) and normalized enrichment scores (NES) against R `fgseaMultilevel` [4]. We observed near-identical ES/NES across a wide range of pathways (Figure 1), achieving rigorous quantitative concordance with the R reference (Pearson $r = 1.000$; Spearman $\rho = 1.000$; RMSE $= 0.0135$; median absolute difference $|\Delta| = 0.0070$ for NES; Figure 1C). ES values were numerically identical within machine precision. Transformed nominal P-values remained overall concordant with the reference implementation (overall Pearson $r = 0.997$; Spearman $\rho = 0.988$; RMSE $= 0.1696$; median $|\Delta| = 0.0467$ on $-\log_{10} P$), although sparse local trajectory windows could show visibly larger dispersion than ES or NES (worst-window Pearson $r = 0.941$ on $-\log_{10} P$; Supplementary Table S7). In such sparse local windows, ES and NES should therefore be regarded as the

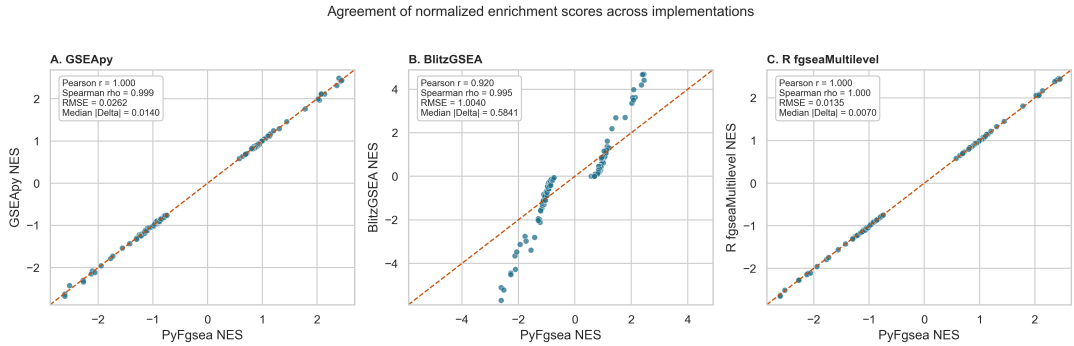

Figure 1 Agreement of normalized enrichment scores (NES) across implementations on matched synthetic benchmark inputs. Panels compare PyFgsea with (A) GSEApy, (B) BlitzGSEA, and (C) R fgseaMultilevel. Dashed lines indicate identity; insets report concordance statistics. PyFgsea is near-identical to the R reference.

primary cross-implementation comparanda, whereas transformed nominal-P agreement is more sensitive to tail-estimation noise and pathway-count filtering. For multilevel P-values, PyFgsea employs a stricter default floor (`eps=1e-50`) compared to R's default (`eps=1e-10`), allowing for higher resolution in extreme tails. When `eps` is matched, deep-tail P-value differences are generally modest, but they become more apparent when few pathways pass size filters or when strongly enriched local windows occupy sensitive rare-event tails (Supplementary Figures S3 and S4). PyFgsea thus preserves ES within machine precision and NES near-identically relative to the reference procedure while offering extended precision for rare events.

3.3. Trajectory case study: rolling-window enrichment

PyFgsea enables rolling-window enrichment along pseudotime for large single-cell datasets. In a real GSE155254-derived HSC-like-to-erythroid trajectory, windowed enrichment identified temporally localized pathway activation patterns while maintaining feasible runtimes over hundreds to thousands of windows (Figure 2). Across rolling-window benchmarks, the stateful runner yielded approximately 1.9-fold speedup in a conservative end-to-end stress test and up to 7.47-fold acceleration in a narrower 100-window component benchmark. Figure 2 uses the default full-window workflow without the optional terminal-truncation extension, which is evaluated separately in Supplementary Figure S14.

4. Discussion

PyFgsea brings an fgseaMultilevel-aligned multilevel estimator to the Python ecosystem and extends it to rolling-window trajectory analysis. When interpreting trajectory dynamics, users should be mindful of the inherent limitations of the rolling-window ranking statistic. By summarizing local expression changes as a simple difference of means, it inherently assumes monotonic expression shifts within the evaluated window. Consequently, genes exhibiting complex, non-monotonic dynamics outside the local window may have their relative changes underestimated, potentially causing false-negative enrichments. Conversely, sparse genes with high technical variance may dominate local rankings, risking false positives. We recommend interpreting dynamic pathways cautiously and cross-referencing with smoothed single-gene visualizations.

Future work includes broader input conventions, additional trajectory ranking statistics, and more diverse cross-platform validation.

Availability and implementation

PyFgsea is released under the MIT License. Source code, documentation, and tutorials are available at

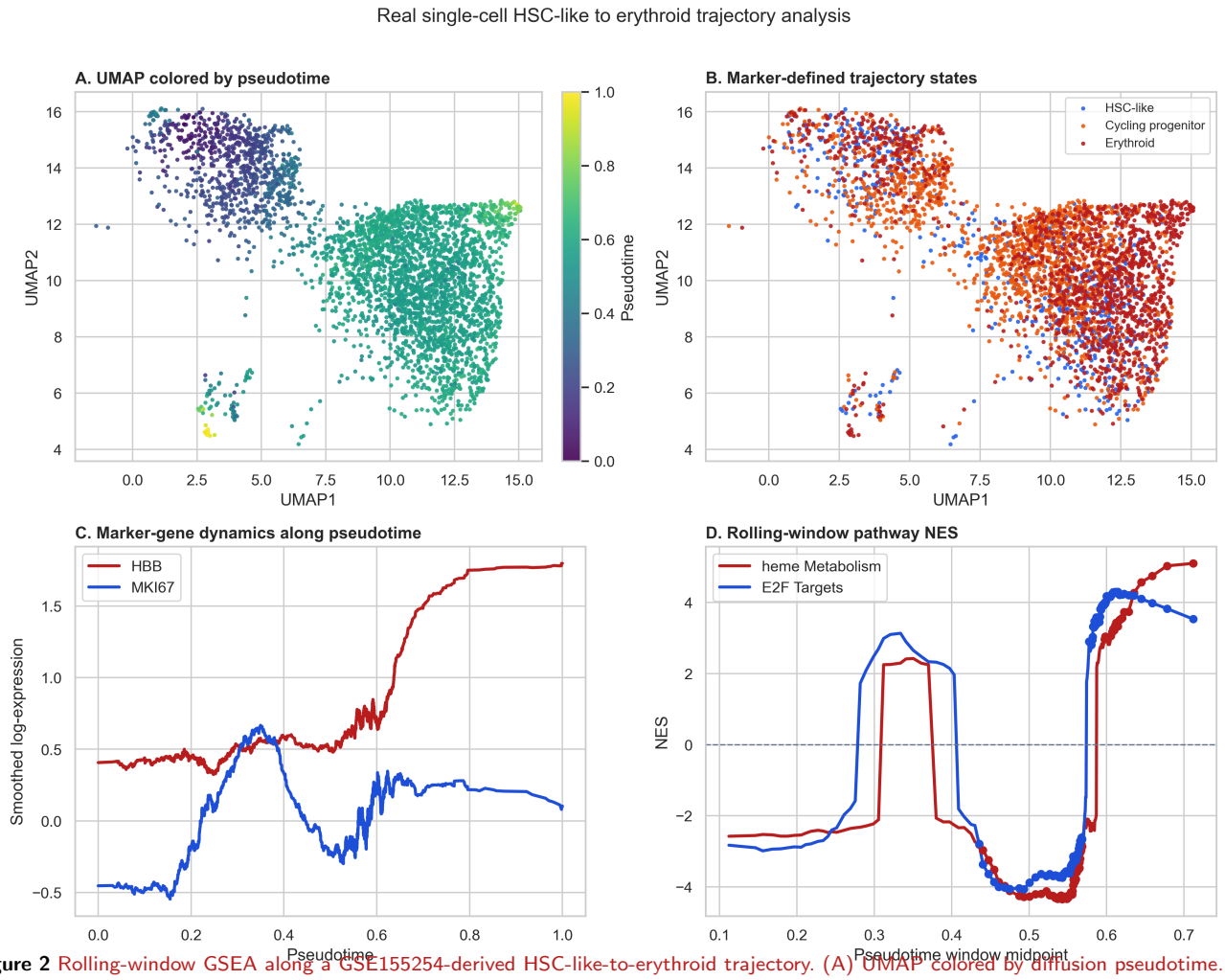

Figure 2 Rolling-window GSEA along a GSE155254-derived HSC-like-to-erythroid trajectory. (A) UMAP colored by diffusion pseudotime. (B) Marker-defined states. (C) Smoothed HBB and MKI67 expression. (D) Window-wise NES for representative pathways; filled points mark within-window FDR < 0.05 after Benjamini–Hochberg correction across pathways. The default full-window workflow is shown; terminal-truncation is audited in Supplementary Figure S14.

https://github.com/shayuanxukuang/pyfgsea. The Python package can be installed directly from PyPI (`pip install pyfgsea`) and supports Linux, Windows, and macOS. A permanent archival snapshot is also available at Zenodo: 10.5281/zenodo.19446446.

Data availability

All scripts and minimal example data required to reproduce the reported software benchmarks and rolling-window workflow are provided in the public PyFgsea repository under `repro/`, `examples/`, and the package source tree (*see Availability and implementation*). Public reference resources used in the showcase analyses include Human Cell Atlas datasets (accessed via `HCAData`) and MSigDB hallmark gene sets; access and licensing follow the respective providers' terms.

Author contributions

K.W. implemented the software and performed experiments. H.S. supervised the project and contributed to manuscript editing. All authors approved the final manuscript.

Conflict of interest

None declared.

Funding

None declared.

References

1. Subramanian A, Tamayo P, Mootha VK, et al. Gene set enrichment analysis: A knowledge-based approach for interpreting genome-wide expression profiles. *Proc Natl Acad Sci U S A*, 102:15545–50, 2005. doi: 10.1073/pnas.0506580102.
2. Fang Z, Liu X, and Peltz G. GSEApy: gene set enrichment analysis in Python. *Bioinformatics*, 39:btac757, 2023. doi: 10.1093/bioinformatics/btac757.
3. Lachmann A, Clarke DJB, Xie Z, et al. blitzGSEA: efficient computation of gene set enrichment analysis through gamma distribution approximation. *Bioinformatics*, 38:2356–7, 2022. doi: 10.1093/bioinformatics/btac076.
4. Korotkevich G, Sukhov V, Budin N, et al. Fast gene set enrichment analysis. *bioRxiv*, 2021. doi: 10.1101/060012. Preprint.
5. Haghverdi L, Büttner M, Wolf FA, et al. Diffusion pseudotime robustly reconstructs lineage branching. *Nat Methods*, 13:845–8, 2016. doi: 10.1038/nmeth.3971.
6. Wolf FA, Angerer P, and Theis FJ. SCANPY: large-scale single-cell gene expression data analysis. *Genome Biol*, 19:15, 2018. doi: 10.1186/s13059-017-1382-0.
7. Liberzon A, Birger C, Thorvaldsdóttir H, et al. The molecular signatures database hallmark gene set collection. *Cell Syst*, 1:417–25, 2015. doi: 10.1016/j.cels.2015.12.004.
8. Liberzon A, Subramanian A, Pinchback R, et al. Molecular signatures database (MSigDB) 3.0. *Bioinformatics*, 27:1739–40, 2011. doi: 10.1093/bioinformatics/btr260.
9. Regev A, Teichmann SA, Lander ES, et al. The human cell atlas. *Elife*, 6:e27041, 2017. doi: 10.7554/eLife.27041.
10. Marini F et al. *HCAData: Accessing the datasets of the Human Cell Atlas in R/Bioconductor*. Bioconductor, 2020. URL `https://bioconductor.org/packages/HCAData`. R/Bioconductor package.